

## Example

```
// returns a list that contains the value added tax amount
// added to the prices of the price list that was passed
param pricelist: queue, VAT: real -> list[real]
var i: integer           // loop index
result.create           // instantiates the results list
i := 1
while not pricelist.empty loop // stop condition
    result[i] := pricelist.pop * (1 + VAT)
    i := i + 1           // increases the index
end                     // end of function
```

See also

[Declare Parameters](#)

## Data Types

## Data Types [SimTalk]

Each *local variable* in *SimTalk* has a data type that defines its range of values and the permitted operations. *Boolean* values, for example, do not support mathematical operations.

## Remarks

*SimTalk* provides these data types:

|                                    |                              |
|------------------------------------|------------------------------|
| Acceleration [SimTalk] - data type | Method [SimTalk] - data type |
| Any [SimTalk] - data type          | Object [SimTalk] - data type |
| Array [SimTalk] - data type        | Queue [SimTalk] - data type  |
| Boolean [SimTalk] - data type      | Real [SimTalk] - data type   |
| Date [SimTalk] - data type         | Speed [SimTalk] - data type  |
| DateTime [SimTalk] - data type     | Stack [SimTalk] - data type  |
| Integer [SimTalk] - data type      | String [SimTalk] - data type |
| JSON [SimTalk] - data type         | Table [SimTalk] - data type  |
| Length [SimTalk] - data type       | Time [SimTalk] - data type   |
| List [SimTalk] - data type         | Weight [SimTalk] - data type |

All of the above data types are also **keywords** in *SimTalk*. You cannot use them to name an object, a local variable, or a function in *SimTalk*.

## Money [SimTalk]

*Plant Simulation* only supports the data type *money* for *global variables*, *user-defined attributes*, and for *lists* and *tables*.

*Money* is not a *SimTalk* data type. For local variables use the data type **real** instead.

## RandTime [SimTalk]

The *global Variable* and the *user-defined attributes* provide the data type *randtime*. You can use it to generate random numbers of data type **time** using a random number stream of its own.

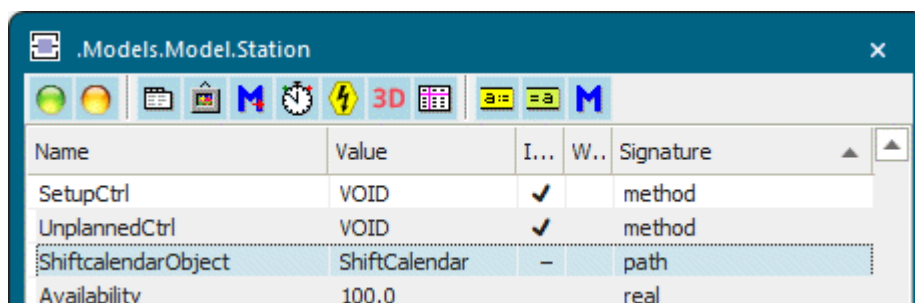
Let's say you want to make the random numbers for the **Set-up Time** independent of the random numbers for the **Processing Time**. To achieve this, create a *user-defined attribute* of data type *randtime*, and set the distribution type of the **Set-up time** to **Formula**, and type `self.<UserDefinedAttribute>.rollDice` into the edit box.

*RandTime* is not a *SimTalk* data type. When getting a *randtime* value, *Plant Simulation* returns a value of data type *time*.

## Path [signature]

The window **Show Attributes and Methods** shows the signature *path* for some objects in the column **Signature**.

*Path* is not a data type that you can assign. If a built-in attribute is listed with the data type *path*, you can assign a value of data type *object* or *string* to it.



| Name                | Value         | I... | W.. | Signature |
|---------------------|---------------|------|-----|-----------|
| SetupCtrl           | VOID          | ✓    |     | method    |
| UnplannedCtrl       | VOID          | ✓    |     | method    |
| ShiftcalendarObject | ShiftCalendar | –    |     | path      |
| Availability        | 100.0         |      |     | real      |

See also

[Data Types in Local Variables](#)

[Keywords](#)

## Acceleration [SimTalk] - data type

A *local variable* of data type *acceleration* has a maximum range of values between  $-8.9 \cdot 10^{307} \leq \text{acceleration} \leq 8.9 \cdot 10^{307}$ , which is displayed like this  $-8.9\text{e}307 \leq \text{real} \leq 8.9\text{e}307$ .

### Remarks

The data type *acceleration* applies to the objects [Conveyor](#) and [Transporter](#).

When assigning it to a variable or an attribute, *Plant Simulation* interprets the value as meters per second squared ( $\text{m/s}^2$ ).

When outputting it, *Plant Simulation* converts the value to the unit you selected under **File > Model Settings/Preferences > Units > Acceleration**.

### Note

In *SimTalk* 2.0 you can specify the acceleration units  $\text{mps}^2$ ,  $\text{cm}^2$ ,  $\text{fps}^2$ , and  $\text{LU/s}^2$ .

Type in the unit directly after the values, without a separating blank space, for example `10mps2` or `10.2mps2`.

To create the  $^2$  symbol, activate the **Num Lock** key on the numeric keypad, hold down **Alt** and type in **0178** on the numeric keypad.

To create the  $^2$  symbol on the **German keyboard layout**, hold down **Alt Gr** and press the key with the number **2**. If your keyboard does not have the **Alt Gr** key, hold down **Ctrl-right+Alt-right** instead.

### Example

```
var a : acceleration := 10mps2 // 10 meters per second squared
var b : acceleration := 2.5fps2 // 2.5 feet per second squared
```

The data types *time*, *length*, *weight*, *speed*, and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real* or *integer* to a variable of data type *length*.

See also

[isAcceleration \[SimTalk\]](#)

## Any [SimTalk] - data type

A *local variable* of data type *any* can take any value.

### Remarks

The data type of the variable assumes the data type of the first assigned value. If you then assign a value that is incompatible with this data type, *Plant Simulation* shows an error.

To reset the assumed data type, use the **forget [SimTalk]** statement.

### Referencing an Object of Type Python

In previous versions, the method *PythonModule.getValue* converted Python data types to *SimTalk* data types where possible. If this was not possible, the Python value was converted to a *string*.

Now *Plant Simulation* returns the Python object as a value of data type *any*.

The Python data type *list* was converted to a *SimTalk* array in previous versions. Now *Plant Simulation* returns the Python list object as a value of data type *any*. This prevents *Plant Simulation* from unnecessarily allocating memory when using large lists.

### Return Value

The return value has the data type *boolean*.

Query the data type of a variable of data type *any* with the following functions. The function expects a parameter whose data type *Plant Simulation* compares.

|                                     |                              |                                          |
|-------------------------------------|------------------------------|------------------------------------------|
| <b>getSimTalkTypename [SimTalk]</b> | <b>isLength [SimTalk]</b>    | <b>typeStatisticsCumulated [SimTalk]</b> |
| <b>isAcceleration [SimTalk]</b>     | <b>isList [SimTalk]</b>      | <b>isString [SimTalk]</b>                |
| <b>isBoolean [SimTalk]</b>          | <b>isListRange [SimTalk]</b> | <b>isTable [SimTalk]</b>                 |
| <b>isDate [SimTalk]</b>             | <b>isObject [SimTalk]</b>    | <b>isTime [SimTalk]</b>                  |
| <b>isDatetime [SimTalk]</b>         | <b>isQueue [SimTalk]</b>     | <b>isWeight [SimTalk]</b>                |
| <b>isInteger [SimTalk]</b>          | <b>isReal [SimTalk]</b>      |                                          |
| <b>isJson [SimTalk]</b>             | <b>isSpeed [SimTalk]</b>     |                                          |

## Examples

```
a := 3.14    -- data type real is assumed
forget a    -- now we can assign a new data type
a := "Test" -- data type string is assumed
```

```
param arg: any // declaring the method named myPrint
if isLength(arg)
  print "length = ", arg
end

// calling the method myPrint
myPrint(1)
myPrint("any text")
myPrint(MyConveyor.length)
```

```
var a : any
a := MyConveyor.acceleration
print a, " ", getUnit(a)
print isAcceleration(a)
```

### Note

The following applies to *isObject*, *isTable*, and *isList*: The value VOID does not have a data type as local variables of data types *object*, *table*, and *list* can all take the value void. For this reason a local variable of data type *any* does not take a data type when you assign void to it. If the function *isObject* is called with such a variable, the function returns the result false.

## Example

```
var a: any := void
print isObject(a)    // prints false
print a = void       // prints true

var b : any := 123
print b = void       // prints false
```

You can also use the function [getSimTalkTypename \[SimTalk\]](#) to get the data type of a variable of data type *any*.

### Return Value

The return value has the data type *string*.

## Example

```
param a : any
switch getSimTalkTypename(a)
case "integer"
  print "integer argument"
case "string"
  print "string argument"
end
```

## See also

### Query the Data Type of a Variable of Data Type Any

## Video on YouTube

<https://youtu.be/5t-wLNmKpbU?si=dR0rGUZ1qLqr3zj7&t=226>

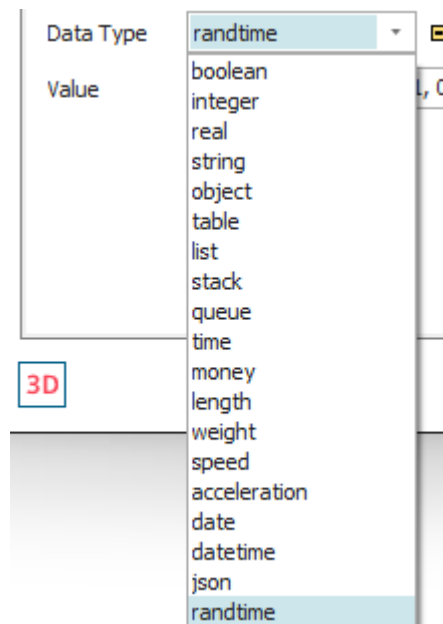
## Array [SimTalk] - data type

### Array [SimTalk] - data type

A local variable of data type *array* designates a one-dimensional or a two-dimensional value field of one of the base data types listed under **Data Types**.

## Remarks

*Base data types* for the *array* data types are:



Base data types can be all data types except for *table*, *list*, *stack*, and *queue*. If you type in *any* as the base data type, each item of the value field can have a different data type. Then, even lists and tables can be placed in the *array*.

Array indexes are one-based, i.e., they start with 1, not with 0.

You can declare *array variables* as follows:

```
a : integer[10]
b : boolean[10,20]
a : string[]
```

- Type in a single number within the brackets to specify a *one-dimensional array* of fixed size, for example `a : integer[10]`.
- Type in two numbers within the brackets to specify a *two-dimensional array* of fixed size, for example `b : boolean[10, 20]`.
- Do not type any number at all within the brackets to specify a *one-dimensional array*, whose size is not fixed, and which is empty initially, for example `a : string[]`.

Compare this example:

```
var vector3 : real[3]           // one-dimensional array with 3 real
values
var matrix3x3 : real[3,3]      // two-dimensional array with 9 real
values
var objList : object[]        // one-dimensional array with n
objects (the size can change)
var a : any[]                 // one dimensional array with n values
of any data type
print vector3[2]              // prints 0 to the console
vector3 := [1.0, 2.0, 3.0]    // fills a one-dimensional array
print vector3[2]              // prints 2 to the console

// For filling a two-dimensional array you might type:
for var x := 1 to matrix3x3.xDim
  for var y := 1 to matrix3x3.yDim
    matrix3x3[x,y] := x + y
  next
next
print matrix3x3               // prints [2, 3, 4][3, 4, 5][4, 5, 6]

print objList.dim              // prints 0 to the console
objList.append(Station)       // adds Station to the array
print objList.dim              // prints 1 to the console
print objList[1]              // prints the path to the Station on the console
a := vector3
a.append("Hello World")
print a.dim                   // prints 4 in the console
print a                       // prints [1, 2, 3, Hello World] to the console
```

*Plant Simulation* displays empty arrays with a space between the brackets like this "[ ]", compare the following example:

```
var a : real[]
var s : string := to_str(a) // s := "[ ]"
```

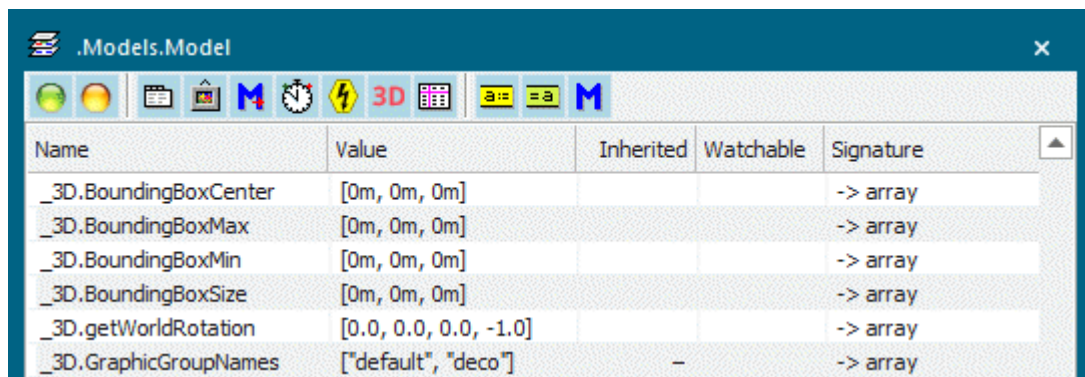
This example creates a *nested array*:

```
-- array of two arrays
-- detect the datatype by the method isArray
var a:any[] := [[current, pi, exp(1)], [Eventcontroller, 42]]
print "a = ",a
print "a[1] = ",a[1]
print "a[2] = ",a[2]

-- datatype array
print isArray(a[1])
print isArray(a)
```

The [Functions for Accessing 3D Objects](#) make extended use of the *array* data types.

The window [Show Attributes and Methods](#) shows the value of an *array* in brackets, compare [Show Methods](#).



| Name                  | Value                 | Inherited | Watchable | Signature |
|-----------------------|-----------------------|-----------|-----------|-----------|
| _3D.BoundingBoxCenter | [0m, 0m, 0m]          |           |           | -> array  |
| _3D.BoundingBoxMax    | [0m, 0m, 0m]          |           |           | -> array  |
| _3D.BoundingBoxMin    | [0m, 0m, 0m]          |           |           | -> array  |
| _3D.BoundingBoxSize   | [0m, 0m, 0m]          |           |           | -> array  |
| _3D.getWorldRotation  | [0.0, 0.0, 0.0, -1.0] |           |           | -> array  |
| _3D.GraphicGroupNames | ["default", "deco"]   | -         |           | -> array  |

Compare the sample models: Click the **Window** ribbon tab, click **Start Page > Getting Started > Example Models > Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.

See also

[Data Types \[SimTalk\]](#)

[Methods and Attributes of Arrays](#)

[isArray \[SimTalk\]](#)



[Print the Contents List to the Console as an Array](#)

[Use a Drag-and-Drop Control for Several Objects](#)

[Video on YouTube](#)

[https://youtu.be/9g6Uou-8eMc?si=wxwJfDmUOR\\_IHVQ5&t=988](https://youtu.be/9g6Uou-8eMc?si=wxwJfDmUOR_IHVQ5&t=988)

## Add and Subtract Arrays of Numerical Data Types

You can add and subtract *arrays* of numerical data types (*integer, real, length, weight, speed, acceleration, time*) whose dimensions match.

### Remarks

In addition, you can multiply *arrays* of a floating point data type (*real, length, weight, speed, acceleration, time*) with a numerical value (*integer, real*) or divide them by a numerical value. We also support **matrix multiplication** for *arrays* of a floating point data type.

### See also

### Relational Operators

## Compare Arrays of Different Data Types

When comparing *arrays* of different data types, *Plant Simulation* throws a runtime error.

### Remarks

*Plant Simulation* throws a runtime error and opens the *Method Debugger* in the following cases:

- *Arrays* of data type *real*, *length*, *time*, *speed*, and *weight* based on their content.
- *Arrays* of data type *any* are compared to other *arrays* based on their content as well.

### Note

*Arrays* of data type *integer* cannot be compared with *arrays* of data type *real*, *length*, *time*, *speed*, or *weight*.

### Examples

```
-- arrays with different data types
var a:any[4]
-- array with different data types
a := [current, pi, -42, [1, 2, 3]]

print "a = ",a
print "a[1]: ", getSimTalkTypeName(a[1])

a[1] := "Hello" -- change the data type
print "a[1]: ", getSimTalkTypeName(a[1])
```

```
-- create, assign, compare arrays
var a:integer[3]

a[1] := 5
a[2] := 2
a[3] := 7
print "a = ",a

var b:integer[3]
print "b = ",b
b := a
print "b = ",b
print "a = b ", a=b
b.sort
print "b = ",b
print "a = b ", a=b
```

See also

## Relational Operators

### Methods and Attributes of Arrays

### Methods and Attributes of Arrays

The *array* data types provide a number of attributes and methods for accessing them.

#### Remarks

Array data types provide these methods and attributes:

|                                                     |                                          |                                        |
|-----------------------------------------------------|------------------------------------------|----------------------------------------|
| <a href="#">append [SimTalk] - array</a>            | <a href="#">deleteValue [SimTalk]</a>    | <a href="#">normalize [SimTalk]</a>    |
| <a href="#">appendArray [SimTalk]</a>               | <a href="#">Dim [SimTalk] - array</a>    | <a href="#">pop [SimTalk] - array</a>  |
| <a href="#">appendValueOfType [SimTalk]</a>         | <a href="#">Empty [SimTalk] - array</a>  | <a href="#">sort [SimTalk] - array</a> |
| <a href="#">asAny [SimTalk]</a>                     | <a href="#">find [SimTalk] - array</a>   | <a href="#">sum [SimTalk] - array</a>  |
| <a href="#">contains [SimTalk] - array</a>          | <a href="#">getValueOfType [SimTalk]</a> | <a href="#">X [SimTalk] - array</a>    |
| <a href="#">copyFromTable [SimTalk]</a>             | <a href="#">insert [SimTalk] - array</a> | <a href="#">xDim [SimTalk] - array</a> |
| <a href="#">copyFromTableColumn [SimTalk]</a>       | <a href="#">join [SimTalk]</a>           | <a href="#">Y [SimTalk] - array</a>    |
| <a href="#">copyToTable [SimTalk]</a>               | <a href="#">Magnitude [SimTalk]</a>      | <a href="#">yDim [SimTalk] - array</a> |
| <a href="#">copyToTableColumn [SimTalk] - array</a> | <a href="#">max [SimTalk] - array</a>    | <a href="#">makeRGBValue [SimTalk]</a> |
| <a href="#">delete [SimTalk] - array</a>            | <a href="#">min [SimTalk] - array</a>    |                                        |

See also

## isArray [SimTalk]

## append [SimTalk] - array

Appends the value designated by the parameter to the end of a *one-dimensional array* designated by <array> that does not have a fixed size.

### Type

### Method

### Syntax

```
<array>.append(Value: any)
```

### Parameter

The parameter *Value* of data type *any* designates the value that you want to append.

### Example

```
var a: string[]  
a.append("string")  
a.append("string")  
print a[2]
```

### See also

[appendArray \[SimTalk\]](#)

[join \[SimTalk\]](#)

## appendArray [SimTalk]

Appends all elements of the array passed as the parameter to the *array* designated by <array>.

### Remarks

The data types of the passed array and the existing array have to match.

*appendArray* only applies to *one-dimensional arrays* which do not have a fixed size.

### Type

### Method

### Syntax

```
<array>.appendArray(ArrayToBeAppended:array)
```

### Parameter

The parameter *ArrayToBeAppended* of data type *array* designates the array that is to be appended to the existing array.

### Example

```
var a : integer[] := [1, 2, 3]
var b : integer[2] := [4, 5]
a.appendArray(b)
print a // outputs [1, 2, 3, 4, 5]
```

### See also

[append \[SimTalk\] - array](#)

## appendValueOfType [SimTalk]

Converts the passed value to a byte sequence and appends these bytes at the end of an *array* of data type *integer* designated by <array> whose size is not defined.

## Remarks

*appendValueOfType* is useful if you want to send binary data with the object [Socket](#).

## Type

## Method

## Syntax

```
<array>.appendValueOfType(Value:any, Type:string)
```

## Parameters

You can specify the following parameters:

- The parameter *Value* of data type *any* designates the value that you want to append.
- The parameter *Type* of data type *string* designates the data type of the value that you want to append.

The passed value can be of data type *integer*, *boolean*, *real*, *length*, *time*, *speed*, *weight*, or *acceleration*. The value is converted to a byte sequence of a defined length where each byte has a value between 0 and 255.

The length depends on the value of the parameter *Type*. Allowed values for *Type* are:

"int8", "int16", "int32", "int64", "uint8", "uint16", "uint32", "uint64", "real32", "real64",  
"Int8", "Int16", "Int32", "Int64", "UInt8", "UInt16", "UInt32", "UInt64", "Real32", and "Real64".

The number in the *Type* designates the number of bits, where 8 bits result in 1 byte. "int16" for example converts the value to 2 bytes and appends these as 2 *integer* values to the *array*.

The byte order of the byte sequence is determined by the first letter of the parameter *Type*.

- If *Type* starts with a lower case letter, the byte sequence is created in little-endian format.
- If *Type* starts with an upper case letter, the byte sequence is created in big-endian format.

## Example

```
var a : integer[]
a.appendValueOfType(123456, "int32") // append 4 bytes
a.appendValueOfType(-3000, "int16")  // append 2 bytes
```

```
var x := a.getValueOfType(1, "int32") // read 4 bytes  
var y := a.getValueOfType(5, "int16") // read 2 bytes
```

See also

[getValueOfType \[SimTalk\]](#)

[Socket](#)



## asAny [SimTalk]

Can be called for *arrays* of data type *string* and *object* designated by <array> and returns a reference to the addressed *array-element*.

## Remarks

*asAny* returns a reference to the addressed *array-element* if its value is a relative or an absolute path to an existing object or to an existing **lane**, **storage place** or to a *user-defined attribute* of data type *table*, *list*, *stack* or *queue*.

## Type

## Method

## Syntax

```
<array>.asAny(Index:integer[, YIndex:integer]) → any
```

## Parameters

You can specify the following parameters:

- The parameter *Index* of data type *integer* designates the index position of the element in a *one-dimensional array*.
- The parameter *Index* designates the X-index-position (i.e., the column number) of the element for a *two-dimensional array*.

The optional parameter *YIndex* of data type *integer* designates the Y-index-position (i.e., the row number) of the element in a *two-dimensional array*.

## Return Value

The return value depends on the type of *array* you specify:

- The return value has the data type *object* if the *array element* contains a reference or a path to an existing object of type *Frame*.
- The return value has the data type *any* if the *array element* contains the path to a **lane** on a *TwoLaneTrack* or to a **storage place**.
- The return value has the data type *table*, *list*, *stack* or *queue* respectively if the *array element* contains the path to a user-defined attribute of data type *table*, *list*, *stack* or *queue*.
- The return value is VOID if the value of the *array element* does not contain a path to a valid object, or something similar.

## Example

```
var a : object[2];  
a[1] := "TwoLaneTrack.A"  
a[2] := "Station.MyTableAttr"  
var obj0 := a[1]           // obj0 is VOID, as not of type 'object'  
var obj1 := a.asAny(1)     // obj1 is .Models.Model.TwoLaneTrack.A of type  
                           // 'any'  
var obj2 := a.asAny(2)     // obj2 is .Models.Model.Station.MeinTabellenAttr  
                           // of type 'table'
```

## See also

[Lane A](#)

[Lane B](#)

[getRoute \[SimTalk\] - Transporter](#)

## contains [SimTalk] - array

Returns if the specified value is contained in the *array* designated by <array> (true) or not (false).

### Remarks

*contains* only applies to *one-dimensional arrays*.

### Type

### Method

### Syntax

```
<array>.contains(Value:any) → boolean
```

### Parameter

The parameter *Value* of data type *any* designates the value which you would like to know if it is contained in the *array*.

### Return Value

The return value has the data type *boolean*.

### Examples

```
var a : string[] := ["hello", "world"]
if a.contains("world")
    print "this code is always reached"
end
if a.contains("hell")
    print "this code is never reached"
end
```

```
param color : string := "yellow"
if ["red","green","blue"].contains(color)
    print "color is red or green or blue"
end
```

## copyFromList [SimTalk]

Copies the list designated by the parameter of data type *list* into the *array* designated by <array>.

### Type

### Method

### Syntax

```
<array>.copyFromList(Source:list) -> void
```

### Parameter

The parameter *Source* of data type *list* designates the list from which you would like to copy the data.

### Return Value

The return value is void.

### Example

```
targetArray.copyFromList(MyList)
```

### See also

[copyToList \[SimTalk\]](#)

## copyFromTable [SimTalk]

Copies the table designated by the parameter of data type *table* into the *array* designated by <array>.

### Remarks

If the *array* has a fixed size, the dimension of the table, or of the range of the table, have to match the size of the *array*.

If you copy the table into a *one-dimensional array*, the values have to be contained in a row of the table.

### Type

Method

### Syntax

```
<array>.copyFromTable( [SourceRange:listrange,] SourceTable:table)
```

### Parameters

You can specify the following parameters:

- The parameter *SourceTable* of data type *table* designates the table from which you would like to copy the data.
- The optional parameter *SourceRange* of data type *listrange* designates the range which you want to copy from the table.

### Example

```
targetArray.copyFromTable(MyDataTable)
```

See also

[copyToTable \[SimTalk\]](#)

## copyFromTableColumn [SimTalk]

Copies the contents of the column of the table into the *one-dimensional array* designated by <array>.

### Type

### Method

### Syntax

```
<array>.copyFromTableColumn(SourceTable:table, Column:integer)
```

### Parameters

You can specify the following parameters:

- The parameter *SourceTable* of data type *table* designates the table from which you would like to copy data.
- The parameter *Column* of data type *integer* designates the column whose data you want to copy.

### Example

```
var array: string[]  
array.copyFromTableColumn(DataTable, 3)
```

## copyToList [SimTalk]

Copies the *array* designated by <array> into a list.

### Type

Method

### Syntax

```
<array>.copyToList(Target:list) -> void
```

### Parameter

The parameter *Target* of data type *list* designates the target list.

### Return Value

The return value is void.

### Example

```
arrayVar.copyToList(MyList)
```

### See also

[copyFromTable \[SimTalk\]](#)

[Write the Content List into a Table for Further Processing](#)

## copyToTable [SimTalk]

Copies the *array* designated by <array> into a table.

### Type

Method

### Syntax

```
<array>.copyToTable(TargetTable:table[, Column:integer, Row:integer])
```

### Parameters

You can specify the following parameters:

- The parameter *TargetTable* of data type *table* designates the target table.
- The optional parameter *Column* of data type *integer* designates the column of the table.
- The optional parameter *Row* of data type *integer* designates the row of the table.

### Example

```
arrayVar.copyToTable(MyDataTable,2,9)
```

### See also

## [copyFromTable \[SimTalk\]](#)

## [Write the Content List into a Table for Further Processing](#)



## copyToTableColumn [SimTalk] - array

Copies the contents of the *one-dimensional array* designated by <array> to the specified column of a table.

### Type

### Method

### Syntax

```
<array>.copyToTableColumn(TargetTable:table, Column:integer)
```

### Parameters

You can specify the following parameters:

- The parameter *TargetTable* of data type *table* designates the table to which you would like to copy data.
- The parameter *Column* of data type *integer* designates the column into which you would like to paste data.

### Example

```
var array: string[2] := ["hello", "world"]  
array.copyToTableColumn(DataTable, 3)
```

## delete [SimTalk] - array

Deletes the contents of the *array* designated by <array>.

### Type

### Method

### Syntax

```
<array>.delete([Index:integer])
```

### Parameter

The optional parameter *Index* of data type *integer* deletes the item located at this index position. The size of the *array* is smaller by 1 after this.

### Example

```
a := [current,pi,e]  
a.delete(2)  
print a.dim
```

**deleteValue [SimTalk]**

Deletes the first occurrence of specified value from the *one-dimensional array* designated by <array>.

**Type**

Method

**Syntax**

```
<array>.deleteValue(Value:any) → boolean
```

**Parameter**

The parameter *Value* of data type *any* designates the value that is going to be deleted.

**Return Value**

The return value has the data type *boolean*.

true if the value is part of the *array*.

false if the value is not part of the *array*.

**Example**

```
a: string[] := ["Alice", "Bob", "Charlie"]  
a.deleteValue("Bob")  
print a  
// prints ["Alice", "Charlie"]
```

## Dim [SimTalk] - array

Returns the dimension of the *one-dimensional array* designated by <array>.

### Type

Read-only attribute

### Syntax

```
<array>.Dim
```

### Return Value

The return value has the data type *integer*.

### Example

```
var a:real[3]  
print a.Dim // prints 3
```

## Empty [SimTalk] - array

Returns if the array designated by <array> is empty (true) or if it contains items (false).

### Remarks

*Empty* only applies to *one-dimensional arrays*.

### Type

Read-only attribute

### Syntax

```
<array>.Empty → boolean
```

### Return Value

The return value has the data type *boolean*.

### Example

```
var a : integer[]  
if a.empty  
  a.append(42)  
end  
if not a.empty  
  print a[1]  
end
```

## find [SimTalk] - array

Searches for a value in the *one-dimensional array* designated by <array>.

### Type

### Method

### Syntax

```
<array>.find(Value:any[, StartIndex:integer])
```

### Parameters

You can specify the following parameters:

- The parameter *Value* of data type *any* designates the value that you want to find.
- The optional parameter *StartIndex* of data type *integer* sets the start index, i.e., the position, from which on *Plant Simulation* starts a search toward the end of the *array*.

### Return Value

The return value is the position of the item which was found.

Is 0 if the value is not contained in the *array*.

### Example

```
var a:real[5] := [1.1, 2.2, 3.3, 4.4, 5.5]
print a.find(3.3)      // prints 3
print a.find(3.3, 4)   // prints 0, that is, not found
```

## getValueType [SimTalk]

Reads a byte sequence from the *array* of data type *integer* designated by <array> and converts it to a value and returns it.

## Remarks

*getValueType* is useful if you want to receive binary data with the object [Socket](#).

## Type

## Method

## Syntax

```
<array>.getValueOfType(Index:integer, Type:string) -> any
```

## Parameters

You can specify the following parameters:

- The parameter *Index* of data type *integer* designates the value that you want to append.  
The parameter *Index* sets at which position in the *array* the byte sequence starts. The length of the byte sequence depends on the value of the parameter *Type*. Allowed values for *Type* are:  
"int8", "int16", "int32", "int64", "uint8", "uint16", "uint32", "uint64", "real32", "real64", "bool",  
"Int8", "Int16", "Int32", "Int64", "UInt8", "UInt16", "UInt32", "UInt64", "Real32", "Real64", and "Bool".
- The parameter *Type* of data type *string* designates the data type of the value that you want to append.  
The number in the *Type* designates the number of bits, where 8 bits result in 1 byte. "int16" for example reads 2 bytes from the *array* and creates an *integer* value between -32768 and 32767.  
The byte order of the byte sequence is determined by the first letter of the parameter *Type*.
  - If *Type* starts with a lower case letter, the byte sequence is interpreted in little-endian format.
  - If *Type* starts with an upper case letter, the byte sequence is interpreted in big-endian format.

## Return Value

The return value has the data type *any*.

## Example

```
var a : integer[]
a.appendValueOfType(123456, "int32") // append 4 bytes
a.appendValueOfType(-3000, "int16")  // append 2 bytes
```

```
var x := a.getValueOfType(1, "int32") // read 4 bytes  
var y := a.getValueOfType(5, "int16") // read 2 bytes
```

See also

[appendValueOfType \[SimTalk\]](#)

[Socket](#)



## insert [SimTalk] - array

Inserts a value into the *one-dimensional array* designated by <array> which does not have a fixed size.

### Type

### Method

### Syntax

```
<array>.insert(Index:integer, Value:any)
```

### Parameters

You can specify the following parameters:

- The parameter *Index* of data type *integer* designates the index position at which *Plant Simulation* inserts the value.
- The parameter *Value* of data type *any* designates the value which you want to insert.

### Example

```
var a: real[] := [1.1, 2.2, 3.3, 4.4, 5.5]  
a.insert(3, -11)  
print a // returns [1.1, 2.2, -11, 3.3, 4.4, 5.5]
```

## join [SimTalk]

Converts the elements within the *array* designated by <array> into a string and unites all elements to a single string. Within the resulting string the elements are separated by the separator string.

### Type

### Method

### Syntax

```
<array>.join(separator:string) → string
```

### Parameter

The parameter *separator* of data type *string* sets the separator string. The separator string can contain any character.

### Return Value

The return value has the data type *string*.

### Example

```
var a:integer[]  
a.append(1)  
a.append(2)  
a.append(3)  
  
print a.join(" * ") // 1 * 2 * 3
```

### See also

[append \[SimTalk\] - array](#)

## Magnitude [SimTalk]

Returns the magnitude/the length of the designated vector of the *numerical one-dimensional array* designated by <array>.

### Type

Read-only attribute

### Syntax

```
<array>.Magnitude → real
```

### Return Value

The return value has the data type *real*.

### Example

```
var direction: real[3] := [2.0, 3.5, 2.8]  
print direction.Magnitude
```

## max [SimTalk] - array

Returns the greatest value contained in the *array* designated by <array>. If the *array* is empty, the method returns 0 for an *array of numbers* and an empty string "" for an *array of strings*.

### Remarks

*max* only applies to *one-dimensional arrays* with a numerical data type or with the data type *string*.

### Type

### Method

### Syntax

```
<array>.max
```

### Return Value

If the method is called with two values having the same physical unit, the returned value also has this unit. Compare these examples:

```
max(time, time)      -> time
max(length, length)  -> length
max(length, speed)   -> real
max(length, real)     -> real
max(integer, integer) -> integer
max(integer, real)    -> integer
max(real, integer)    -> integer
max(integer, weight)  -> real
```

### Example

```
var a:real[5] := [1.1, 2.2, 3.3, 4.4, 5.5]
print a.max // prints 5.5
```

```
var arrPartPos: length[]
arrPartPos := [6300mm, 5600mm]
var maxPartPos: length := arrPartPos.max
print arrPartPos.max
```

### See also

[min \[SimTalk\] - array](#)

## min [SimTalk] - array

Returns the smallest value contained in the *array* designated by <array>.

### Remarks

*min* only applies to *one-dimensional arrays* with a numerical data type or with the data type *string*.

### Type

### Method

### Syntax

```
<array>.min
```

### Return Value

If the method is called with two values having the same physical unit, the returned value also has this unit. Compare these examples:

```
min(time, time)      -> time
min(length, length)  -> length
min(length, speed)   -> real
min(length, real)     -> real
min(integer, integer) -> integer
min(integer, real)    -> integer
min(real, integer)    -> integer
min(integer, weight)  -> real
```

### Example

```
var a:real[5] := [1.1, 2.2, 3.3, 4.4, 5.5]
print a.min // prints 1.1
```

```
var arrPartPos:length[]
arrPartPos := [6300mm, 5600mm]
print arrPartPos.min
```

### See also

[max \[SimTalk\] - array](#)

**normalize [SimTalk]**

Normalizes the *array* designated by <array> of a numerical data type.

**Remarks**

*normalize* only applies to *numerical one-dimensional arrays*.

A normalized vector retains its direction, its length (its *Magnitude*) becomes 1 though. The resulting vector is often called unit vector. A vector is normalized by dividing it by its own length.

**Type**

Method

**Syntax**

```
<array>.normalize → real
```

**Return Value**

The return value has the data type *real*.

Is the magnitude of the designated old vector.

**Example**

```
var v:real[3]
v := [2,3,4]
print v.normalize
print v
// prints 5.3851648071345
[0.371390676354104, 0.557086014531156, 0.742781352708207]
```

**See also**

[Magnitude \[SimTalk\]](#)

## pop [SimTalk] - array

Removes the last element from the *array* designated by <array> and returns this value.

### Type

Method

### Syntax

```
<array>.pop → any
```

### Return Value

The return value has the data type *any*.

### Example

```
var a:integer[]  
a.append(1)  
a.append(2)  
a.append(3)  
print a.pop // 3
```

## sort [SimTalk] - array

Sorts the *array* designated by <array>.

### Remarks

When sorting a *two-dimensional array*, *Plant Simulation* sorts the rows according to the first column.

#### Note

Sorting is case-sensitive.

### Type

Method

### Syntax

```
<array>.sort(Descending:boolean:=false)
```

### Parameter

The optional parameter *Descending* of data type *boolean* sets how the array is going to be sorted:

- Specify `true` to sort the array in **descending** order.
- Specify `false` or do not specify the parameter to sort the array in **ascending** order.

#### Default Value of the Parameter

The default value is `false`.

### Examples

```
var a:integer[] := [4, 2, 1, 3]
a.sort(false) // ascending
print a       // returns [1, 2, 3, 4]
```

```
var a:integer[] := [4, 2, 1, 3]
a.sort(true) // descending
print a      // returns [4, 3, 2, 1]
```



## sum [SimTalk] - array

Returns the sum of all values contained in the *array* designated by <array>.

### Remarks

*sum* only applies to *one-dimensional arrays*.

### Type

### Method

### Syntax

```
<array>.sum
```

### Example

```
var a:real[5] := [1.1, 2.2, 3.3, 4.4, 5.5]  
print a.sum // returns 16.5
```

## X [SimTalk] - array

Returns the element of the *array* designated by <array> with the index 1.

### Remarks

*X* applies to *one-dimensional arrays*, which are of data type *integer*, *real*, or *length*, and have a fixed dimension of **three elements**. The attribute *X* points to the element of the *array* with the index 1.

*X* applies to *one-dimensional arrays*, which are of data type *integer*, *real*, or *length*, and have a fixed dimension of **two elements**.

### Type

Attribute

### Syntax

```
<array>.X -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
var c: real[3] := Station.Coordinate3D
print c.X // prints c[1]
c.X := 42 // c[1] := 42
```

### See also

[Y \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

## xDim [SimTalk] - array

Returns the x-dimension of the *two-dimensional array* designated by <array>.

### Type

Method

### Syntax

```
<array>.xDim -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
var matrix:real[2,3]  
print matrix.xDim // prints 2
```

## Y [SimTalk] - array

Returns the element of the *array* designated by <array> with the index 2.

### Remarks

Y applies to *one-dimensional arrays*, which are of data type *integer*, *real*, or *length*, and have a fixed dimension of **three elements**. The attribute Y points to the element of the *array* with the index 2.

Y applies to *one-dimensional arrays*, which are of data type *integer*, *real*, or *length*, and have a fixed dimension of **two elements**.

### Type

Attribute

### Syntax

```
<array>.Y -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
var c: real[3] := Station.Coordinate3D
print c.Y // prints c[2]
c.Y := 42 // c[2] := 42
```

### See also

[X \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

## yDim [SimTalk] - array

Returns the y-dimension of a *two-dimensional array* designated by <array>.

### Remarks

*yDim* returns 0 for a *one-dimensional array*. This way you can find out if a local variable of data type *any* contains a *one-dimensional array* or a *two-dimensional array* and if it was assigned to this station, not if it actually was there yet.

### Type

Method

### Syntax

```
<array>.yDim -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
var matrix:real[2, 3]  
print matrix.yDim // prints 3
```

## Z [SimTalk] - array

Returns the element of the *array* designated by <array> with the index 3.

### Remarks

Z applies to *one-dimensional arrays*, which are of data type *integer*, *real*, or *length* and have a fixed dimension of **three elements**.

### Type

Attribute

### Syntax

```
<array>.Z -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
var c: real[3] := Station.Coordinate3D
print c.Z // prints c[3]
c.Z := 42 // c[3] := 42
```

### See also

[X \[SimTalk\] - array](#)

[Y \[SimTalk\] - array](#)

## Boolean [SimTalk] - data type

A local variable of data type *boolean* can be `true` or `false`.

See also

[Boolean \[SimTalk\] - value](#)

[isBoolean \[SimTalk\]](#)

## Date [SimTalk] - data type

A *local variable* of data type *date* designates a date between 01.01.1900 and 31.12.9999.

### Remarks

When assigning a value to a variable or an attribute of the data type *time*, you have to convert the date with the function [str\\_to\\_date](#).

Before you assign a value to the local variable, it has an *initial value*. It is 0 for all *numerical* data types, false for the data type *boolean*, an empty string ("") for the data type *string*, and void for the data types *object* and *list* or *table*.

The *initial value* for the data types *date* and [DateTime](#) depends on the *Plant Simulation* version in which you initially created your simulation model. For models, which you created in *Plant Simulation* 14.0 or higher, the start value is 01.01.1900.

### See also

[isDate \[SimTalk\]](#)



## DateTime [SimTalk] - data type

A *local variable* of data type *dateTime* designates a date and a time between 01.01.1900 00:00:00 and 31.12.9999 23:59:59.

### Remarks

When assigning a value to a variable or an attribute, you have to convert the date with the function [str\\_to\\_dateTime](#).

Before you assign a value to the local variable, it has an *initial value*. It is 0 for all *numerical* data types, false for the data type *boolean*, an empty string ("") for the data type *string*, and void for the data types *object* and *list* or *table*.

The *initial value* for the data types [Date](#) and *dateTime* depends on the *Plant Simulation* version in which you initially created your simulation model. For models, which you created in *Plant Simulation* 14.0 or higher, the start value is 01.01.1900.

### See also

[isDatetime \[SimTalk\]](#)

## Integer [SimTalk] - data type

A *local variable* of data type *integer* can only contain integer numbers.

### Remarks

The range of values is between  $-9.223.372.036.854.775.808 \leq \text{integer} \leq 9.223.372.036.854.775.807$ .  
*Plant Simulation* does not check for values outside of this range.

### See also

### Integer [SimTalk] - value

### isInteger [SimTalk]

## JSON [SimTalk] - data type

### JSON [SimTalk] - data type

A *local variable* of data type JSON can hold values in the JSON data format.

### Remarks

JSON, standing for **JavaScript Object Notation**, is a standardized data format that can be used for exchanging data between applications. Many web applications and web services expect their input data and return their results in JSON format.

JSON data is structured data that can be nested arbitrarily. The data consists of one, none, or several elements. Each element is a name-value-pair. The name can contain spaces and special characters.

#### Note

Names in JSON are case-sensitive.

The values of a name-value-pair can have the following data types:

- *Integer*: A signed integer number
- *Real*: A floating point number
- *Boolean*: true or false
- *String*: A string within quotation marks, for example "my string"
- *Array*: An ordered list of values within brackets [ ] and separated by commas
- *JSON*: An unordered list of elements (name-value-pairs) within braces { } and separated by commas

- *Void*: An empty value that JSON displays as `null`

```
var person: json
person["Name"] := "Robert"
person["Age"] := 42
person["Personnel No"] := "007-1234-55"
```

```
var person
person["age"] := 30
person["age"] += 1 // Congratulations :-)
```

You can also **address a name-value-pair via an integer index**. This way you can iterate across all name-value-pairs. You always have to either select the name or the value from such a name-value-pair by appending `.Name` or `.Value` respectively.

```
var j: json
j["color"] := "red"
j["shape"] := "triangle"
for var i := 1 to j.dim
  print j[i].Name, " ", j[i].Value
next
```

For more information about JSON, consult Wikipedia <https://en.wikipedia.org/wiki/JSON>.

See also

[Elements of Data Type Object in JSON Variables](#)

[\\_Methods and Attributes of JSON](#)

[isJson \[SimTalk\]](#)

## JSON Literals

JSON literals are enclosed in curly braces { } and contain JSON elements separated by commas. The JSON elements consist of a string literal, a colon, and a JSON value.

### Remarks

JSON values can be any of the following:

- A number (an integer literal or a real literal)
- A string literal
- The boolean literals true and false
- void
- A local variable of data type *integer*, *real*, *boolean*, *string*, *object*, *array*, or *json*

### Example

```
var j: json
var jobTitle: string := "Engineer"
var addressArray: string[] := ["Main street 42", "10001 New York"]
j := { "name": "Alice", "age": 42, "female": true,
      "job": jobTitle, "address": addressArray }
```

### See also

### [Why Use SimTalk 2.0?](#)

## Elements of Data Type Object in JSON Variables

JSON variables can, among others, accept elements of data type *object*.

### Remarks

```
var j: json
j["o"] := MyStation    -- element "o" is of data type object
j["o"].Label := "ABC"  -- assigns a label to the object MyStation
```

### Note

JSON, standing for **J**ava**S**cript **O**bject **N**otation, is a standardized data format that can be used for exchanging data between applications.

The **element type object** is an extension that is not defined within the JSON standard and which only *Plant Simulation* provides. When JSON data is converted into a *string*, for example to send it via an interface, all object elements will be written as a *string* with their absolute path to the *object*. This is necessary so that the created *string* is a standard-conform representation which other applications can process.

```
var json1, json2: json
json1["o"] := MyStation    -- element "o" is of data type object
var s : string := to_str(json1) -- convert data into a string
json2.parse(s)             -- element "o" is of data type string
```

## Methods and Attributes of JSON

### \_Methods and Attributes of JSON

The data type *json* provides the methods and attributes listed in the table of contents for accessing it.

### Remarks

*JSON* provides these methods and attributes:

|                                    |                            |
|------------------------------------|----------------------------|
| asString [SimTalk] - JSON          | Dim [SimTalk] - JSON       |
| contains [SimTalk] - JSON          | getOrCreateJSON [SimTalk]  |
| copy [SimTalk] - JSON              | parse [SimTalk]            |
| copyToTableColumn [SimTalk] - JSON | readFile [SimTalk] - JSON  |
| copyToTableRow [SimTalk] - JSON    | unshareAndDelete [SimTalk] |
| delete [SimTalk] - JSON            | writeFile [SimTalk] - JSON |

**Note**

You can also **address a name-value-pair via an integer index**. This way you can iterate across all name-value-pairs.

You always have to either select the name or the value from such a name-value-pair by appending `.Name` or `.Value` respectively.

```
var j: json
  j["color"] := "red"
  j["shape"] := "triangle"
  for var i := 1 to j.dim
    print j[i].Name, " ", j[i].Value
  next
```

## asString [SimTalk] - JSON

Returns the string representation of the JSON object designated by <json>.

### Remarks

You can, for example, send the returned string to a Web-server. The method *parse* can re-transform such a string into the JSON format.

### Type

Method

### Syntax

```
<json>.asString([MultiLine:boolean:=true]) -> string
```

### Parameter

The optional parameter *MultiLine* of data type *boolean* sets if the method returns the result with line breaks and indenting spaces (*true*) or without them (*false*).

### Default Value of the Parameter

The default value is *true*.

### Return Value

The return value has the data type *string*.

### Example

```
var j: json
j["Name"] := "Tom"
j["Age"] := 32
var s: string := j.asString(false)
```

### See also

[parse \[SimTalk\]](#)

## contains [SimTalk] - JSON

Returns if the JSON object designated by <json> contains a name-value-pair with the passed name (true) or not (false).

### Type

### Method

### Syntax

```
<json>.contains(Name:string) -> boolean
```

### Parameter

The parameter *Name* of data type *string* designates the name which you would like to know if it is contained in the JSON object.

### Return Value

The return value has the data type *boolean*.

### Example

```
var j: json
j["Body color"] := "blue"
if a.contains("Body color")
    print "This code will always be reached."
end
```



## copy [SimTalk] - JSON

Returns a copy of the JSON object designated by <json>.

## Type

Method

## Syntax

```
<json>.copy -> json
```

## Return Value

The return value has the data type *json*.

## Examples

```
var Person1, Person2: json
Person2 := Person1      -- Person1 and Person2 reference the same JSON
object
Person2["Name"] := "Alice" -- also changes Person1["Name"]
print Person1["Name"]    -- returns Alice

Person2 := Person1.copy  -- Person1 and Person2 now are different JSON
objects
Person2["Name"] := "Bob"  -- does not change Person1["Name"]
print Person1["Name"]    -- returns Alice
```

```
var Person, Job: json
Job["Description"] := "engineer"
Person["Job"] := Job.copy    -- inserts a copy of Job in Person
Job["Company"] := "Siemens"  -- does not change Person["Job"]
```

## copyToTableColumn [SimTalk] - JSON

Copies the contents of the *json array* designated by <json array> to the specified column of a table.

### Type

Method

### Syntax

```
<json array>.copyToTableColumn(TargetTable:table, Column:integer)
```

### Parameters

You can specify the following parameters:

- The parameter *TargetTable* of data type *table* designates the table to which you would like to copy data.
- The parameter *Column* of data type *integer* designates the column into which you would like to paste data.

### Example

```
var j:json
j["a"] := ["hello", "world", 42]
j["a"].copyToTableColumn(DataTable, 2)
-- copies the three array elements into the first
-- three cells of the second column of the table
```

## copyToTableRow [SimTalk] - JSON

Copies the contents of the *json array* designated by <json array> to the specified row of a table.

### Type

### Method

### Syntax

```
<json array>.copyToTableRow(TargetTable:table, Row:integer)
```

### Parameters

You can specify the following parameters:

- The parameter *TargetTable* of data type *table* designates the table to which you would like to copy data.
- The parameter *Row* of data type *integer* designates the row into which you would like to paste data.

### Example

```
var j:json
j["a"] := ["hello", "world", 42]
j["a"].copyToTableRow(DataTable, 2)
-- copies the three array elements into the first
-- three cells of the second row of the table
```

## delete [SimTalk] - JSON

The method *delete* without parameter deletes the entire contents of the JSON object designated by `<json>`. The method *delete* with parameter deletes the specified name-value pair.

### Type

Method

### Syntax

```
<json>.delete([Name:string]) -> boolean
```

### Parameter

The optional parameter *Name* of data type *string* deletes the name-value-pair with the specified name.

### Return Value

The return value has the data type *boolean*.

true if the JSON object did contain the name before.

false if it did not contain the name before.

### Example

```
var person: json
person["Name"] := "Jason"

if person.delete("name")
  print "This code will never be reached."
elseif person.delete("Name")
  print "This code will always be reached."
end
```

## Dim [SimTalk] - JSON

Returns the number of name-value-pairs which the JSON object designated by <json> contains.

### Type

Read-only attribute

### Syntax

```
<json>.Dim -> integer
```

### Return Value

The return value has the data type *integer*.

### Example

```
param s: string
var j: json
j.parse(s)
if j.Dim = 0
    print "No data available."
end
```

**getOrCreateJSON [SimTalk]**

Returns a reference to the empty JSON object designated by <json>.

**Remarks**

If the JSON object does not contain a name-value-pair with the specified name yet in the variable, *Plant Simulation* creates a new name-value-pair and sets an empty JSON object as value. The method *getOrCreateJSON* returns a reference to this empty JSON object.

If the JSON object designated already contains a name-value-pair with the specified name yet in the variable and the value has the data type *json*, the method *getOrCreateJSON* returns a reference to the contained value. If the value has another data type than *json*, *Plant Simulation* shows an error.

**Type**

Method

**Syntax**

```
<json_Variable>.getOrCreateJSON(Name:string) -> json
```

**Parameter**

The parameter *Name* of data type *string* designates the name of the name-value-pair.

**Return Value**

The return value has the data type *json*.

Is a reference to the empty JSON object.

Is a reference to the value contained in the JSON object.

**Example**

```
-- source code of Method1:
var JsonObj: json

var ColorInfo := JsonObj.getOrCreateJSON("ColorInfo")
-- JSON-subobject named "ColorInfo" will be created and returned
ColorInfo["color"] := "red"
ColorInfo["NumberChangesColor"] += 1

Method2(JsonObj)

-- source code of Method2:
```

```
param JsonObj: json

var ColorInfo := JsonObj.getOrCreateJSON("ColorInfo")
-- existing JSON-subobject will be returned
ColorInfo["color"] := "green"
ColorInfo["NumberChangesColor"] += 1
-- the value will be increased to 2
```

## parse [SimTalk]

Parses the text passed by the parameter *Data* and returns the parsed value.

### Remarks

The text usually starts with an opening curly brace { and specifies a complex JSON object, although the value can also be a single value of data type *integer*, *real*, *boolean*, or an *array*.

If the value is a JSON object, this value will also be assigned to the variable <json>, otherwise the variable <json> will be set to an empty JSON object.

### Type

Method

### Syntax

```
<json>.parse(Data:string) -> any
```

### Parameter

The parameter *Data* of data type *string* designates the text to be parsed.

### Return Value

The return value has the data type *any*.

### Examples

```
var j: json

var value1 := j.parse("{ \"count\": 123 }")
// value1: json = { "count": 123 }
// j      : json = { "count": 123 }
```

```
var value2 := j.parse("123")
// value2: integer = 123
// j      : json = {}
```

```
var arr := j.parse("[3.141, 100, true]")
// arr   : any[] = [3.141, 100, true]
// j      : json = {}
```



[parse \[SimTalk\]](#)

**See also**

[asString \[SimTalk\] - JSON](#)

## readFile [SimTalk] - JSON

Imports the JSON source code from the specified text file into the specified variable of data type JSON.

### Type

### Method

### Syntax

```
<json_Variable>.readFile(FileName:string) → void
```

### Parameter

The parameter *FileName* of data type *string* designates the path to and the name of the text file containing the source code you want to use.

### Return Value

The return value is *void*.

### Example

```
var js:json  
js.readFile("D:\json.txt")  
print js
```

### SimTalk

## [writeFile \[SimTalk\] - JSON](#)

## unshareAndDelete [SimTalk]

Unshares the reference to the JSON object designated by <json> and assigns an empty JSON object to the *local variable* or to the *user-defined attribute*.

### Type

### Method

### Syntax

```
<json>.unshareAndDelete
```

### Example

```
var person1, person2 : json
person2 := person1      -- person1 and person2 reference the same JSON
object
person2["Name"] := "Alice" -- also changes person1["Name"]
person2.unshareAndDelete -- person2 is an empty JSON object
person2["Name"] := "Bob"  -- only changes person2["Bob"]
```

## writeFile [SimTalk] - JSON

Writes the JSON source code from the specified JSON variable to the specified text file.

### Type

Method

### Syntax

```
<json_Variable>.writeFile(FileName:string) → void
```

### Parameter

The parameter *FileName* of data type *string* designates the path to and the name of the text file to which the source code is going to be written.

### Return Value

The return value is *void*.

### Example

```
-- our text file JSON.txt contains this JSON source code:
{
  "first_name": "John",
  "last_name": "Doe",
  "age": 46,
  "address": {
    "street_address": "42 42nd Street",
    "city": "New York",
    "state": "NY",
    "postal_code": "10021-3100"
  }
}
```

### SimTalk

## [readFile \[SimTalk\] - JSON](#)

## Length [SimTalk] - data type

A *local variable* of data type *length* has a maximum range of values between  $-8.9 \cdot 10^{307} \leq \text{length} \leq 8.9 \cdot 10^{307}$ , which is displayed like this  $-8.9\text{e}307 \leq \text{real} \leq 8.9\text{e}307$ .

### Remarks

When assigning it to a variable or an attribute, *Plant Simulation* interprets the value as meters (m).

When outputting the value to the *Debugger* or the *Console*, *Plant Simulation* converts the value to the unit you selected under **File > Model Settings/Preferences > Units > Length**.

### Note

In *SimTalk* 2.0 you can specify the length units m, mm, km, cm, yd, ft, and in.

Type in the unit directly after the values, without a separating blank space, for example 10m or 10.2m.

You can specify the unit for *floating point values* and for *integer values*.

The data types *time*, *length*, *weight*, *speed* and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real* or *integer* to a variable of data type *length*.

### Example

```
var len := 1.0ft
var s : speed := 10.5m / 1:30
var x : length := 3m
```

### See also

[isLength \[SimTalk\]](#)

## List [SimTalk] - data type

A *local variable* of data type *list* shares its built-in properties with the object *DataList*.

### Remarks

Note the difference between the object **DataList**, which you insert into your model, and the data type *list*. You can create *user-defined attributes* and *local/global variables* of data type *list* that are part of another object and thus are not an object of their own and do not have their own icon.

For this reason variables and attributes of data type *list* do not recognize all of the attributes, read-only attributes, and methods of the object *DataList*, such as **Location** or **existsIcon**. All other methods, especially for read and write access, apply to both the *DataList* and to variables and attributes of data type *list*.

### Note

Before accessing the data type *list* for the first time, you have to create the *local variable* with the method **create** or you have to assign a value to the variable.

### Example

```
var l: list[string]
l.create
l.insert(1, "Hello")
```

### See also

[Instantiating Local Lists and Tables](#)

[Data Types in Local Variables](#)

[create \[SimTalk\] - local list](#)

[insert \[SimTalk\] - lists](#)

[delete \[SimTalk\] - lists](#)

[isListRange \[SimTalk\]](#)

[isList \[SimTalk\]](#)

## Method [SimTalk] - data type

A *local variable* of data type *method* is a *user-defined attribute* of data type *method*, also called an *attribute method* in most cases.

### Remarks

This *user-defined attribute* is a part of the object, i.e., it moves with the object, when you insert it into another *Frame* or model or when you move it. Compare the topics [Entrance control](#) and [self](#) for details about using it.

The object [Method](#) and *user-defined attributes* of data type *method* have their own random number stream which you can assign with the attribute [RandomSeed](#).

As you are used to from the material flow objects, *Plant Simulation* automatically assigns the random number stream when you insert an object, so that each object has its own random number stream.

Method execution for *user-defined attributes* of data type *method*, which are deleted while they are being executed, is terminated immediately, it will not be continued. Compare the following examples:

- Suppose that a *MU* has a *user-defined attribute* of data type *method* which was suspended via a **waituntil-instruction**. Then, the *MU* is deleted in the *Drain*, which naturally also deletes its *user-defined attributes*. Thereupon the *Method* is deleted from the list of suspended methods and method execution is terminated, i.e., the instructions after the **waituntil-instruction** will not be executed.

If the *attribute method* was called from another *Method*, execution of this *Method* will be continued.

If the *attribute method* had a return value, the value, which the implicitly existing local variable *result* had at this point in time, will be returned to the calling *Method*.

- Suppose that a *MU* has a *user-defined attribute* of data type *method* which is called by the **Entrance Control** of a *Station*. The *attribute method* in turn calls a *Method* that is inserted in the *Frame*. This *Method* deletes the *MU* with the statement `Station.Cont.delete`.

The execution of the *attribute method* and of the *Method* called by it are terminated immediately. The return value of the *attribute method* (or VOID if no return value was declared) will be returned to the **Entrance Control** whose execution will be continued.

*User-defined attributes* of data type *method* and objects of type *Method* behave the same when you extend the path. For the instruction `Station.MyMethAttr.executeIn(60)` *Plant Simulation* calls the *user-defined attribute* named *MyMethAttr* and not the method `executeIn(60)`.

If you want to prevent the *user-defined attribute* from being executed, you have to use the [reference operator &](#), `Station.&MyMethAttr.executeIn(60)` for our example above.

See also

[RandomSeed \[SimTalk\] - Method/User-defined Attribute](#)

## User-defined Attributes [general description]



## Object [SimTalk] - data type

A *local variable* of data type *object* points to an object in your simulation model or is *void*.

### Remarks

You can save any **graphically** represented object with the data type *object*. This variable can be a local variable, a formal parameter, a global variable and values in tables.

You can assign both objects and *strings* to the variables of data type *object* mentioned above.

If you assign an object, *Plant Simulation* creates an **Object Reference**, regardless if you specify the object with a relative or an absolute **path**.

Specify the object with a relative path to immediately resolve the relative path within the context of the current *Method* and create an object reference to the computed object.

Let's suppose that the method *.Models.Model.Method* contains the following method call:

```
SubFrame.methodU(Station)
```

*Plant Simulation* resolves the relative path **Station** to an object reference to *.Models.Model.Station* and passes this object reference to the method within the *SubFrame*. If the path cannot be resolved to an object, *Plant Simulation* opens the *Debugger*.

If a *string* is passed to a variable, the *string* is assigned as a path to the variable. Regardless if this path is relative or absolute, *Plant Simulation* does not resolve it at this point in time. Instead, the path is resolved each and every time when the variable is read.

Let's suppose that the method *.Models.Model.Method* contains the following method call:

```
SubFrame.methodU("Station")
```

*Plant Simulation* passes the relative path *Station* to the method within the *SubFrame*. At this point in time such an object neither has to exist within the context of the calling method, nor within the context of the called method as the path is not resolved yet.

Let's suppose that we inserted an object of type *Station* into the *SubFrame* and that this object has the name *Station1*.

Let's also suppose that the source code of *methodU* in the *SubFrame* looks like this:

```
param o: object
Station1.Name := "Station"
```

```
o.name := "Station1"
o.name := "Test" // ERROR!
```

Each time the *local variable* named *o* is accessed, the relative path will be resolved, in fact it will be resolved within the context of this method, meaning that path resolution starts in the *SubFrame*. This way it points to *.Models.Model.SubFrame.Station*, which now exists. This *Station* is then renamed to *Station1* so that the path *Station* cannot be resolved in the next line of code and *Plant Simulation* thus opens the *Debugger*.

You can assign values of data type *object* to variables of data type *string* or to attributes of data type *string*. If the value contains an **object reference**, the absolute path will be assigned with a leading asterisk (\*). If the value contains a relative or an absolute path, this path is assigned.

```
var o1: object := "Station"
var o2: object := ".Models.Model.Station"
var o3: object := .Models.Model.Station
var s1: string := o1 -- "Station"
var s2: string := o2 -- ".Models.Model.Station"
var s3: string := o3 -- ".*Models.Model.Station"
```

See also

[isObject \[SimTalk\]](#)

[The Absolute Path](#)

[The Relative Path](#)

[Object Reference](#)

## Queue [SimTalk] - data type

A *local variable* of data type *queue* shares some of the built-in properties of the object *DataQueue*.

### Remarks

Note the difference between the object **DataQueue**, which you insert into your model, and the data type *queue*. You can create *user-defined attributes* and *local/global variables* of data type *queue* that are part of another object and thus are not an object of their own and do not have their own icon.

For this reason variables and attributes of data type *queue* do not recognize all of the attributes, read-only attributes, and methods of the object *DataQueue*, such as **Location** or **existsIcon**. All other methods, especially for read and write access, apply to both the *DataQueue* and to variables and attributes of data type *queue*.

### Note

Before accessing the data type *queue* for the first time, you have to create the *local variable* with the method **create** or you have to assign a value to the variable.

### Example

```
var q: queue[string]
q.create
q.push("Hello")
q.delete
-- deletes the contents from the cell of the queue
```

### See also

[Instantiating Local Lists and Tables](#)

[Data Types in Local Variables](#)

[create \[SimTalk\] - local list](#)

[push \[SimTalk\] - Stack](#)

[delete \[SimTalk\] - lists](#)

[isQueue \[SimTalk\]](#)

## Real [SimTalk] - data type

A *local variable* of data type *real* contains floating point numbers. The value range is between  $-1.7976931348623158 \times 10^{308}$  and  $1.7976931348623158 \times 10^{308}$ .

### Remarks

Real numbers have a precision of 16 places (meaning digits, i.e., digits before and after the decimal point). When displaying the numbers, the computer rounds to 15 places.

As floating point numbers can only save a fixed number of places, rounding errors can, in principle, always occur when computing with floating point numbers. For this reason you should, as a rule, not use the equal operator `=` when comparing floating point numbers as it checks for exact equivalence. Use the about-equal-operator `~=` instead.

You can also type in floating point numbers with mantissa and exponent in this notation: `1.0e-3` this corresponds to `0.001`.

Internally *Plant Simulation* always saves the numbers with mantissa and exponent. The accuracy of the mantissa is limited to 16 decimal digits. This is determined by the representation of floating point numbers in the CPU, which is defined by the IEEE Standard for Floating-Point Arithmetic (IEEE 754).

The CPU can only process binary data, i.e., zeros and ones. For this reason floating point numbers are represented according to the IEEE-754 standard in the binary numeral system. Consequently, the mantissa actually does not contain 16 decimal digits but 53 binary numbers. Many numbers, which can be represented within the decimal system with a finite number of digits, cannot be represented with a finite number of digits in the binary numeral system.

### Examples

Compare these examples:

- The decimal number **1.125** can be represented precise as  $1 + 0/2 + 0/4 + 1/8$ . This corresponds to the binary number `1.001`.
- The decimal number **1.1** cannot be represented precise with a finite number of places. Within the binary numeral system this would be  $1 + 0/2 + 0/4 + 0/8 + 1/16 + 1/32 + 0/64 + 0/128 + 1/256 + 1/512 + 0/1024 + 0/2048 + 1/4096 + 1/8192 + \dots$

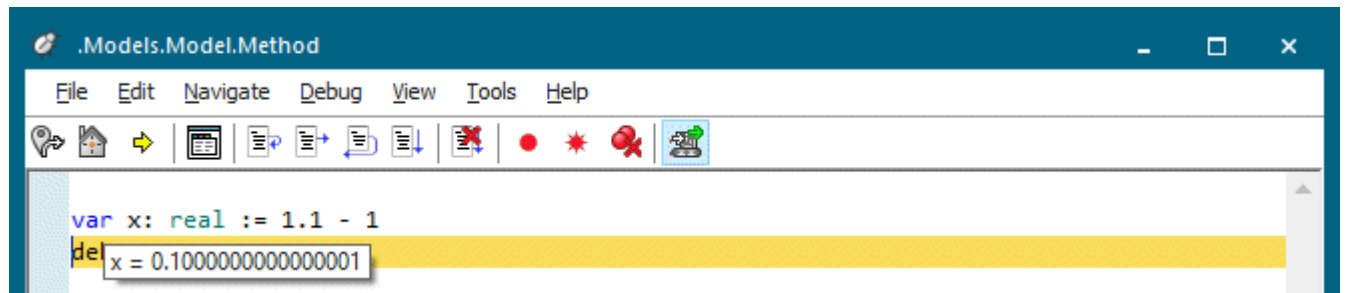
This corresponds to the binary number `1.0001100110011...`

If you type a number with decimal point and digits after the decimal point into the *Method Editor*, the computer converts this number to a floating point number according to the IEEE-754 standard when interpreting the source code so that the CPU can process it. The number is thus converted into a binary number with a fixed number of places. If binary places are cut off during this process, this number minimally differs from the decimal number that you specified.

You can view this effect when you run the following *SimTalk* instruction:

```
var x: real := 1.1 - 1
```

You can then see in the *Method-Debugger* that x has the value **0.10000000000000001**.



See also

[isReal \[SimTalk\]](#)

[Real \[SimTalk\] - value](#)

## Speed [SimTalk] - data type

A *local variable* of data type *speed* has a maximum range of values between  $-8.9 \cdot 10^{307} \leq \text{speed} \leq 8.9 \cdot 10^{307}$  which is displayed like this  $-8.9\text{e}307 \leq \text{real} \leq 8.9\text{e}307$ .

### Remarks

When assigning it to a variable or an attribute, *Plant Simulation* interprets the value as meters per second (m/s). When outputting it, *Plant Simulation* converts the value to the unit you selected under **File > Model Settings/Preferences > Units > Speed**.

### Note

In *SimTalk* 2.0 you can specify the speed units mps, fps, kmh, and mph.

Type in the unit directly after the number, without a separating blank space, for example 100kmh or 100.5kmh.

```
var len := 1.0ft
var s : speed := 10.5m / 1:30
var x : length := 3m
```

The data types *time*, *length*, *weight*, *speed* and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real*, or *integer* to a variable of data type *length*.

### See also

### isSpeed [SimTalk]

## Stack [SimTalk] - data type

A *local variable* of data type *stack* shares some of the built-in properties of the object *DataStack*.

### Remarks

Note the difference between the object **DataStack**, which you insert into your model, and the data type *stack*. You can create *user-defined attributes* and *local/global variables* of data type *stack* that are part of another object and thus are not an object of their own and do not have their own icon.

For this reason variables and attributes of data type *stack* do not recognize all of the attributes, read-only attributes, and methods of the object *DataStack*, such as **Location** or **existsIcon**. All other methods, especially for read and write access, apply to both the *DataStack* and to variables and attributes of data type *stack*.

### Note

Before accessing the data type *list* for the first time, you have to create the *local variable* with the method **create** or you have to assign a value to the variable.

### Example

```
var s: stack[string]
s.create
s.push("bottles")
-- adds the cell containing the string bottles as the
-- first cell of the variable of data type stack
s.delete
-- deletes the contents from the cell in the stack
```

See also

[Instantiating Local Lists and Tables](#)

[Data Types in Local Variables](#)

[create \[SimTalk\] - local list](#)

[push \[SimTalk\] - Stack](#)

[delete \[SimTalk\] - lists](#)

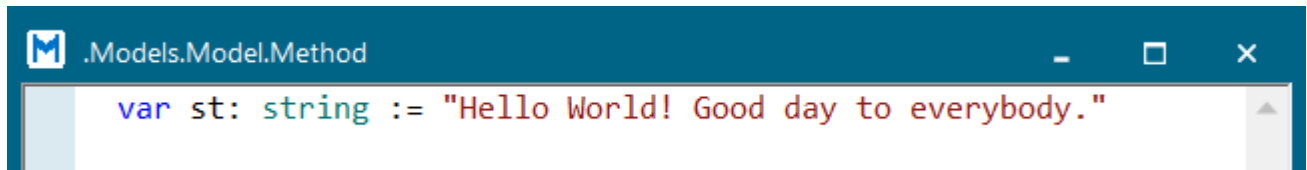
[isStack \[SimTalk\]](#)

## String [SimTalk] - data type

A *local variable* of data type *string* can consist of any number of characters, meaning upper or lower case letters, numbers, special characters, etc. enclosed in quotation marks `"`.

### Remarks

*Plant Simulation* shows a value of data type *string* in the source code in this color `"test"`.



```
var st: string := "Hello World! Good day to everybody."
```

### See also

#### Functions for Editing Variables of Data Type String

Compare strings with [relational operators](#)

#### Colors for Syntax Highlighting

#### isString [SimTalk]

#### String [SimTalk] - value



## Table [SimTalk] - data type

A *local variable* of data type *table* shares some of the built-in properties of the object *DataTable*.

### Remarks

Note the difference between the object **DataTable** that you can insert into models and the data type *table*. You can create *user-defined attributes* and *local/global variables* of data type *table* that are part of another object and thus are not an object of their own and do not have their own icon.

For this reason variables and attributes of data type *table* do not recognize all of the attributes, read-only attributes, and methods of the object *DataTable*, for example **Location** or **existsIcon**. All other methods, especially for read and write access apply to both the object *DataTable* and to variables and attributes of data type *table*.

### Note

Before accessing the data type *table* for the first time, you have to create the *local variable* with the method **create** or you have to assign a value to the variable.

### Examples

```
var OrderList: table[string,real]
OrderList.create
OrderList[1,1] := "Cans"
OrderList[2,1] := 3000.0
OrderList.delete
```

```
var myWaitingTimesTable: table
MyAssembly.statWaitingTimeTable(myWaitingTimesTable)
```

### See also

[Instantiating Local Lists and Tables](#)

[Data Types in Local Variables](#)

[create \[SimTalk\] - local list](#)

[insert \[SimTalk\] - lists](#)

[delete \[SimTalk\] - lists](#)

[isTable \[SimTalk\]](#)

## Time [SimTalk] - data type

A *local variable* of data type *time* has a maximum range of values between  $-8.9 \cdot 10^{307} \leq \text{time} \leq 8.9 \cdot 10^{307}$  which is displayed like this  $-8.9\text{e}307 \leq \text{real} \leq 8.9\text{e}307$ .

### Remarks

When assigning it to a variable or an attribute, the value is always interpreted in seconds (sec). *Plant Simulation* converts the output to a compound format with hours (hh) minutes (mm) and seconds (ss.ss) separated by colons: hh:mm:ss.ss

The data types *time*, *length*, *weight*, *speed*, and *acceleration* are not compatible! You can, for example, only assign a value of data type *length*, *real*, or *integer* to a variable of data type *length*.

### See also

[isTime \[SimTalk\]](#)